CAN Analyzer USB Protocol – Detailed Specification

---

1. Introduction

This system allows a PC application (UI) to communicate with a CAN hardware device over USB.

It performs:

- Connecting to CAN hardware
- Configuring CAN settings
- Sending CAN messages
- Receiving CAN messages
- Monitoring system health

---

2. USB CONNECT FRAME (PC → DEVICE)

Frame Structure

Byte 0 : Command

Byte 1 : Interface

Byte 2 : Channel

Byte 3 : Nominal Baud Rate

Byte 4 : Mode / Data Baud

Byte 5 : CAN Type + Flags

Byte 6 : Reserved

CONNECT FRAME – Byte-by-Byte Explanation

Byte	Name	Description	Possible Values
Byte 0	Command	Defines the operation	`0xA0` = CONNECT
Byte 1	Interface	Communication interface used	`0x01` = USB
Byte 2	Channel	Selects CAN channel/port	`0x00` = Channel 1 `0x01` = Channel 2
Byte 3	Nominal Baud Rate	CAN arbitration speed	`0x01` = 10 kbps `0x02` = 20 kbps `0x03` = 50 kbps `0x04` = 80 kbps `0x05` = 100 kbps `0x06` = 125 kbps `0x07` = 250 kbps `0x08` = 500 kbps `0x09` = 800 kbps `0x0A` = 1 Mbps
Byte 4	Mode / Data Baud	Depends on CAN type	**Classic CAN:** `0x00` = Normal `0x01` = Loopback `0x02` = Silent **CAN FD:** `0x01` = 1 Mbps `0x02` = 2 Mbps `0x03` = 4 Mbps `0x04` = 5 Mbps `0x05` = 8 Mbps

Byte 5	CAN Type + Flags	Bitfield configuration	Bit 0 → `0` Classic / `1` FD Bit 1 → BRS Bit 2 → ISO / Non-ISO Bit 3–7 → Reserved
Byte 6	Reserved	Future use	Always `0x00`

Byte 5 (Flags) – Detailed Bit Table

Bit	Name	Description	Values
Bit 0	CAN Type	Select CAN mode	`0` = Classic CAN `1` = CAN FD
Bit 1	BRS	Bit Rate Switching	`0` = OFF `1` = ON
Bit 2	FD Format	CAN FD format type	`0` = ISO `1` = Non-ISO
Bit 3–7	Reserved	Future use	Set to `0`

Examples

Classic CAN (500 kbps, Normal Mode)

A0 01 00 08 00 00 00

Classic CAN (250 kbps, Loopback)

A0 01 00 07 01 00 00

CAN FD (500k / 2M, BRS ON)

A0 01 00 08 02 03 00

Example

Frame: A0 01 00 08 02 03 00

Byte	Value	Meaning
0	A0	CONNECT
1	01	USB
2	00	Channel 0
3	08	500 kbps
4	02	2 Mbps (FD Data Baud)
5	03	FD + BRS
6	00	Reserved

---

3. CONNECT RESPONSE (DEVICE → PC)

Frame Structure

Byte 0 : 0xA1 (ACK)

Byte 1 : Status

Byte 2 : Channel

Byte 3 : CAN Type

Byte 4 : Reserved

Byte-by-Byte Explanation

Byte	Name	Description	Possible Values
Byte 0	Command	Response identifier for CONNECT	`0xA1` = ACK (Acknowledgement)
Byte 1	Status	Indicates the result of the CONNECT request	`0x00` = Success `0x01` = Failure `0x02` = Invalid Baud `0x03` = Hardware Error
Byte 2	Channel	Channel number that was configured	`0x00` = Channel 1 `0x01` = Channel 2
Byte 3	CAN Type	Indicates active CAN mode	`0x00` = Classic CAN `0x01` = CAN FD
Byte 4	Reserved	Reserved for future use	Always `0x00`

Status Field (Byte 1)

Code	Name	Description
0x00	SUCCESS	Connection established successfully
0x01	FAILURE	General failure (unspecified error)
0x02	INVALID BAUD	Unsupported baud rate received
0x03	HARDWARE ERROR	CAN controller or hardware failed

Example Frames

Success (Classic CAN, Channel 0): A1 00 00 00 00

Byte	Value	Meaning
0	A1	ACK
1	00	Success
2	00	Channel 0
3	00	Classic CAN
4	00	Reserved

Success (CAN FD, Channel 1): A1 00 01 01 00

Byte	Value	Meaning
0	A1	ACK
1	00	Success
2	01	Channel 1
3	01	CAN FD
4	00	Reserved

Error (Invalid Baud Rate)

Byte	Value	Meaning
0	A1	ACK
1	02	Invalid Baud
2	00	Channel 0
3	00	Classic CAN (default/fallback)
4	00	Reserved

---

4. CAN RECEIVE FRAME (DEVICE → PC)

Frame Structure

Byte 0 : 0xF1 (Prefix)

Byte 1 : 0x00 (RX Frame)

Byte 2–5 : Timestamp (4 bytes)

Byte 6–9 : CAN ID

Byte 10 : DLC + Channel

Byte 11–N : Data

Byte-by-Byte Explanation

Byte	Name	Description	Possible Values
Byte 0	Prefix	Start of CAN frame marker	`0xF1` (always)
Byte 1	Frame Type	Indicates type of frame	`0x00` = RX (Received CAN frame)
Byte 2–5	Timestamp	Time when frame was received (32-bit, little-endian)	`0x00000000` – `0xFFFFFFFF`
Byte 6–9	CAN ID	CAN Identifier (Standard or Extended)	11-bit or 29-bit ID Bit 31 = 1 → Extended
Byte 10	DLC + Channel	Data length + Channel number	Lower 4 bits = DLC Upper 4 bits = Channel
Byte 11–N	Data	Actual CAN payload data	Classic: 0–8 bytes FD: 0–64 bytes

CAN RX FRAME – Detailed Field Breakdown

Bytes 2–5: Timestamp

Property	Description
Size	4 bytes (uint32)
Format	Little-endian
Unit	Microseconds or milliseconds (implementation dependent)

Bytes 6–9: CAN ID

Field	Description
Bits 0–28	CAN Identifier
Bit 31	Extended ID flag `0` = Standard ID (11-bit) `0` = Extended ID (29-bit)

Byte 10: DLC + Channel

Bit Breakdown:

Bits	Field	Description
0–3	DLC	Data Length Code
4–7	Channel	CAN Channel

DLC Mapping Table:

DLC (Binary) (Bits 0–3)	DLC (Hex)	Actual Data Length
0000	0x0	0 bytes
0001	0x1	1 byte
0010	0x2	2 bytes
0011	0x3	3 bytes

0100	0x4	4 bytes
0101	0x5	5 bytes
0110	0x6	6 bytes
0111	0x7	7 bytes
1000	0x8	8 bytes
1001	0x9	12 bytes
1010	0xA	16 bytes
1011	0xB	20 bytes
1100	0xC	24 bytes
1101	0xD	32 bytes
1110	0xE	48 bytes
1111	0xF	64 bytes

Channel Mapping Table:

Channel	**Binary (Bits 7–4)**	**Hex Value (Upper value)**	**Full Byte Example (with DLC=8)**
Channel 1	0000	0x0	0000 1000 → 0x08
Channel 2	0001	0x1	0001 1000 → 0x18

---

5. CAN TRANSMIT FRAME (PC → DEVICE)

Frame Structure

Byte 0 : 0xF1

Byte 1 : 0x01 (TX Frame)

Byte 2–5 : CAN ID

Byte 6 : DLC + Channel

Byte 7–N : Data

Byte-by-Byte Explanation

Byte	Name	Description	Possible Values
Byte 0	Prefix	Start of CAN frame	`0xF1` (always)
Byte 1	Frame Type	Indicates TX frame	`0x01` = Transmit CAN frame
Byte 2–5	CAN ID	Identifier of CAN message	11-bit (Standard) 29-bit (Extended) Bit31 = 1 → Extended
Byte 6	DLC + Channel	Data length + Channel number	Upper 4 bits = Channel Lower 4 bits = DLC
Byte 7–N	Data	CAN payload to transmit	Classic: 0–8 bytes FD: 0–64 bytes

Example 1: Classic CAN TX Frame

Frame: F1 01 21 03 00 00 08 AA BB CC DD EE FF 00 11

Field	Value	Meaning
Prefix	F1	Start
Type	01	TX
CAN ID	21 03 00 00	0x321
DLC+CH	08	DLC=8, CH=0
Data	AA–11	8 bytes

Example 2: CAN FD TX Frame

Frame: F1 01 21 03 00 80 19 11 22 33 44 55 66 77 88 99 AA BB CC

Field	Value	Meaning
Prefix	F1	Start
Type	01	TX
CAN ID	21 03 00 80	Extended ID
DLC+CH	19	DLC=9, CH=1
Data	11–CC	12 bytes (FD mapping)

---

6. USB HEARTBEAT (CONNECTION CHECK)

Request (PC → Device)

Frame Structure

Byte 0 : 0xD0 (ALIVE Request)

Byte 1 : 0x00 (Reserved)

Byte-by-Byte Explanation

Byte	Name	Description	Possible Values
Byte 0	Command	Heartbeat request	`0xD0`
Byte 1	Reserved	Future use	`0x00`

Response (Device → PC)

Frame Structure

Byte 0 : 0xD1 (ALIVE Response)

Byte 1 : Status

Byte-by-Byte Explanation

Byte	Name	Description	Possible Values
Byte 0	Command	Heartbeat response	`0xD1`
Byte 1	Status	General health	`0x00` = OK `0x01` = Error

Timing

Parameter	Value	Description
Send Interval	1 second	PC sends heartbeat
Response Time	< 50 ms	Device must respond quickly
Timeout	3 seconds	No response → disconnect
Retry Count	3	Max missed responses

Connection Logic - PC Side

Condition	Action
Response received	Connection OK
No response	Increment timeout counter
Timeout reached	Mark disconnected

Connection Logic - Device Side

Condition	Action
Receive 0xD0	Send 0xD1 immediately
CAN error	Update status field
Idle state	Report via state field

7. SYSTEM BEHAVIOR

Condition	Meaning
USB OK + CAN frames	Running
USB OK + no frames	Bus Idle
USB lost	Disconnected
CAN error	Fault

FINAL SUMMARY

This protocol defines:

✔ USB connection handling
✔ CAN configuration (Classic + FD)
✔ CAN frame TX/RX
✔ Heartbeat monitoring
